Systèmes multiagents Orientée-Objet

Programmation Orientée Objet, ENSIMAG 2A - 2025

OUYAHIA Iles, MONTAURIOL Nicolas,
JOUINI Youssef, DANG Duong Dang Khoa, BASSO Marcelo

1 Introduction

Dans ce rapport, nous aborderons **les choix d'implémentation** (avec les UML initiaux et finaux), les classes créées, les tests réalisés et les résultats obtenus pour le projet « Systèmes multiagents Orienté-Objet ». Le projet consiste en la conception des 5 jeux suivants, selon les paradigmes de l'orientation objet et les bonnes pratiques d'implémentation dans la langage de programmation Java : i) **Les balles** ii) **Le jeu de la vie de Conway** iii) **Le jeu de l'immigration** iv) **Le modèle de Schelling** v) **Les boids**.

2 Choix de Conception

2.1 Conception initiale

Nous avons commencé le projet à partir de la construction conceptuelle des classes et des héritages, en factorisant les paramètres et les méthodes communes aux classes supérieures. De plus, le polymorphisme est au centre de notre conception : lors du draw(), le Simulator appelle chaque entité qui se « dessine » elle même. L'UML de notre projet initial se trouve dans les annexes Figure 3 (Le code .puml des diagrammes peut être consulté dans le dossier /rapport/assets/).

Dans un premier temps, nous ne sommes pas entré dans les détails de la construction des « Boids », cependant nous avons préparé la structure afin d'accueillir ce simulateur. La conception était divisé en 5 packages :

1. **UI** - Gestion des fenêtres
2. **Core** - Simulateur central avec les principales méthodes et paramètres, qui se divise en
   - **GridSimulator** - Simulateur avec grille pour automates cellulaires
   - **SpaceSimulator** - Simulateur avec espace continu pour Balls et Boids
3. **Systems** - Package contenant les simulateurs de chaque jeu
   - toutes les jeux...
4. **Entities** - Package avec toutes les entités prévues
   - **Cell** - Pour les automates cellulaires
   - **Particle** - Pour les Balls et les Boids
5. **World** - Package avec les propriétés des environnements des jeux
   - **Grid** - géré par GridSimulator
   - **Space** - géré par SpaceSimulator

Nous avons défini ces divisions car, à partir du travail demandé, il a été possible d'identifier plusieurs propriétés factorisables. Par exemple :

1. Une classe **Entities.Entity** centrale
2. Une classe **World.Board** centrale avec les differents environnements des jeux
3. Une classe abstraite **Core.Simulator** centrale, interfacé par le Simulable, avec les méthodes/paramètres :
   (a) **next(), restart(), update()**
   (b) **Board** et **Entity** - responsable de la gestion du Board (environnement du jeu) et des entités.
   (c) **draw()** - responsable d'appeler la fonction draw() du Board et de chaque Entité.

4. des classes dediés aux jeux qui orchestrent les calculs telles que : **Systems.BoidsSimulator** ou la classe abstraite **Systems.GridSimulator**, qui sert de base aux trois jeux d'automates cellulaires, qui sont très similaires. La seule différence entre Conway et Immigration, par exemple, réside dans le nombre d'états et la fonction update().
5. Une classe **Entities.Cell** pour les 3 jeux d'automates cellulaires.
6. Une classe **Entities.Particle** pour les entité de Balls et Boids.

2.2 Développement et conception finale

Au cours du développement, nous avons réalisé que nous pouvions simplifier la conception et supprimer les classes inutiles. Tout au long du projet, nous avons utilisé les tests pour orienté ou modifier nos choix de conception (cf. tests). Cependant, la structure du projet n'a que peu variée, quelques changements de méthodes, de classes et d'héritage. Le résultat de tous ces changements (diagramme final) est visible à la Figure 4

2.3 Balls

Pour la mise en œuvre de « Balls », nous avons suivi les instructions fournies dans la description du projet. Balls utilise la classe « Space » (même classe que pour « Boids ») comme « Board ». L'espace du jeu imite un environnement contigu, mais utilise des coordonnées entières.

Question 3. Dans quelles classes sont invoquées les méthodes de dessin ? le simulateur principal (Simulator) implemente la fonction draw(). Pour cela, il itère sur chaque entité qui, en realité, se dessine elle même (polymorphisme).

2.3.1 UI et fenêtres

L'interface utilisateur principale du système (visible lorsque vous exécutez la commande « make ») a été conçue pour faciliter le choix du simulateur. Elle est construite à partir des bibliothèques *Java AWT, Java Swing et gui* (JDK >= v.17) (fournie).

Elle crée une fenêtre avec différents boutons et, lorsque vous cliquez dessus, elle lance le simulateur choisi et exécute son interface graphique. Ensuite, l'utilisateur peut changer de simulateur en cliquant simplement sur un autre bouton, car l'interface se charge de fermer le jeu actuel et de lancer l'exécution du nouveau simulateur.

2.3.2 Automates celulaires

Tous les jeux automatiques mobiles sont une extension de la classe **« GridSimulator »**. Ils ont été factorisés et les règles des jeux ont été implémentées dans les fonctions « update() » de chaque simulateur. « Conway » et « Immigration » sont fondamentalement le même jeu, avec un nombre differente d'états.

Sur « SchellingSimulator », la fonction « update() » a une implémentation plus complexe, car il y a 2 état mais chaque état peut avour c couleurs et il existe un nombre plus grand de restritions. L'attribution d'une nouvelle cell aux familles qui déménagent se fait de manière aléatoire dans la grille, ce qui rend la simulation plus aléatoire et plus réaliste, grace à la fonction .

Dans les implémentations Grid, nous utilisons la **structure de données Array<Entity>** car, en interprétant une matrice comme un tableau, nous avons la possibilité d'un accès en O(1) et les avantages de la localité spatiale dans l'accès à la mémoire, ce qui rend cette représentation optimale pour parcourir toutes les cellules.

Question 7. À partir de quel seuil obtenez-vous une ségrégation ? Au fur et à mesure que K augmente, la simulation termine plus vite (converge plus facilement). Par contre, pour couleurs, si le nombre de couleurs augmentent, on observe l'effet inverse. Il y a aussi un paramètre "coverage" qui determine le nombre de cellules à occuper. En ce que concerne le "coverage", lors qu'il augmente, la simulation prends plus de temps pour converger.

2.3.3 Boids

Notre implémentation des Boids comporte plusieurs détails intéressants, mais il est important de souligner que le fonctionnement des Boids est décrit par seulement quelques fonctions dans la classe *Boid.java* e *BoidsSimulator.java*. Cependant, plusieurs petits comportements sont implémentés dans de petites méthodes afin de rendre le jeu plus fluide et réaliste. Par exemple, lorsque un Boid s'approche d'un prédateur, il augmente sa vitesse en acquérant une accélération de fuite (panique) ; l'angle de virage est également limité dans une fenêtre, afin d'éviter qu'une entité ne tourne de manière irréaliste.

Une autre fonctionnalité implémentée est celle des « locker » pour la chasse. Lorsqu'un prédateur aperçoit une proie, il définit un verrou d'objectif et tente de rester cohérent avec sa victime afin d'éviter toute indécision de la part des prédateurs.

L'une des principales fonctionnalités mises en œuvre est la capacité du jeu à **s'adapter à n boids**. Dans le cadre de ce travail, il fallait seulement un prédateur et une proie. Cette vision a conduit à l'ajout d'une variable booléenne dans la structure Boid appelée « prédateur ». Cependant, en insérant un troisième Boid (Octopos), nous avons commencé à remarquer des problèmes dans cette conception (en plus d'un viol evident des principes de l'orientation objet). Dans un exemple simple avec 3 boids : a, b et c, si a prédate b, b prédate c et c prédate a, qui est le prédateur de qui ? Il n'est pas possible de résoudre ce problème avec une variable booléenne. Nous avons donc introduit un **facteur de prédation**. Ainsi, lorsque 2 boids se rencontrent, leurs facteurs de prédation sont comparés et la relation est définie.

L'indice est calculé dans la fonction **predatorScore()** dans le fichier Boid.java. Il se base sur d'autres critères du boid, tels que « fear », « hunt » et « hunger ». La valeur finale est calculée en multipliant raw = c*hunt + k*fear, où c et k sont des constantes. Ensuite, on calcule raw*hunger, et on renvoit la valeur obtenu. En plus de ça modularité, cette conception est totallement adapté au principe POO.

Les boids ont aussi des **affinités sociales** (capacité à former des groupes et à rester ou non alignés avec eux). Chaque boid peut configurer ses facteurs individuellement et ceux-ci sont divisés en 3 : alignement, cohésion et séparation. Ces propriétés dictent les comportements sociaux de chaque Boid et définissent les règles de formation et de séparation des groupes.

De plus, les boids ont un **panicRadius**, qui est une sensibilité à la panique autour d'eux. Cette fonctionnalité permet, si un banc s'approche d'un danger, à l'un des poissons de « communiquer » sa peur aux autres par des mouvements brusques et de rediriger le reste des boids pour créer un effet "meute".

Pour cette implémentation, nous utilisons une simple liste d'entité templatée selon le jeu (pour Grid : ArrayList<Cell>) qui permet d'accéder à chaque entité en un temps constant. Pour faciliter la suppression des Boids nous utilisons la **structure de données HashSet<Boid>**.

Pour créer les images « Shark » et « Fish », nous avons utilisé la bibliothèque open source *svg2java*, crée par Bernd Rosstauscher. Ainsi, nous pouvons utiliser les logos « Boids » comme un objet Java, en définissant plus simplement des propriétés telles que l'angle et la couleur, ce que optimise grandement le performance. Les fichiers générés ont été enregistrés dans le dossier « utils/img/ ».

3 Tests effectués et résultats

Pour les tests, nous avons utilisé les bibliothèques : *junit-jupiter-api-5.14.0*, *junit-platform-console-standalone-1.12.2* et *junit-4.13.1*.

Nous avons créé des batteries de tests pour les entités : 1. **Cell** 2. **Boid**, et pour chacun des simulateurs. Au total, nous avons créé **264 tests unitaires**. L'utilisateur peut lancer tous les tests avec la commande « make test » ou choisir un simulateur spécifique, par exemple « make test-conway », « make test-immigration », ...

Après la création des tests, des changements importants ont été apportés à « Boids », ce qui a entraîné plusieurs défaillances. Les tests ont permis de détecter ces changements et de corriger leur portée afin qu'ils n'altèrent pas le fonctionnement des autres systèmes. De plus, les tests eux-mêmes ont été modifiés afin de s'adapter aux changements valides apportés à la structure de « Boids » Figure 1.

L'intégration de tests automatisés dans le projet a permis de valider, à chaque étape, le système dans son ensemble, garantissant ainsi que les nouvelles implémentations n'affectaient pas les anciennes. De plus, cela a permis de détecter et de corriger les bugs.

Nous avons également configuré des tests de régression sur la plateforme GitLab. Ainsi, après chaque « push » effectué dans le référentiel, les tests sont exécutés en pipeline et nous avertissent en cas d'erreur. Figure 2

Annexe



FIGURE 1 – Description des erreurs de test après changement sur Boids.

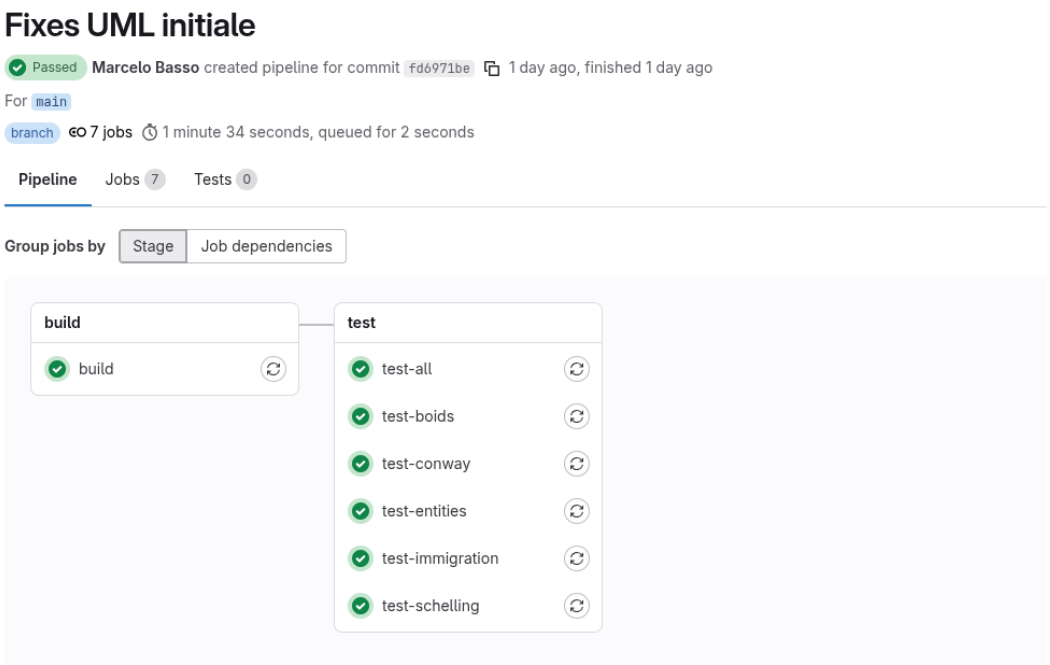


FIGURE 2 – Photo de la pipeline Gitlab pour les test de regression.

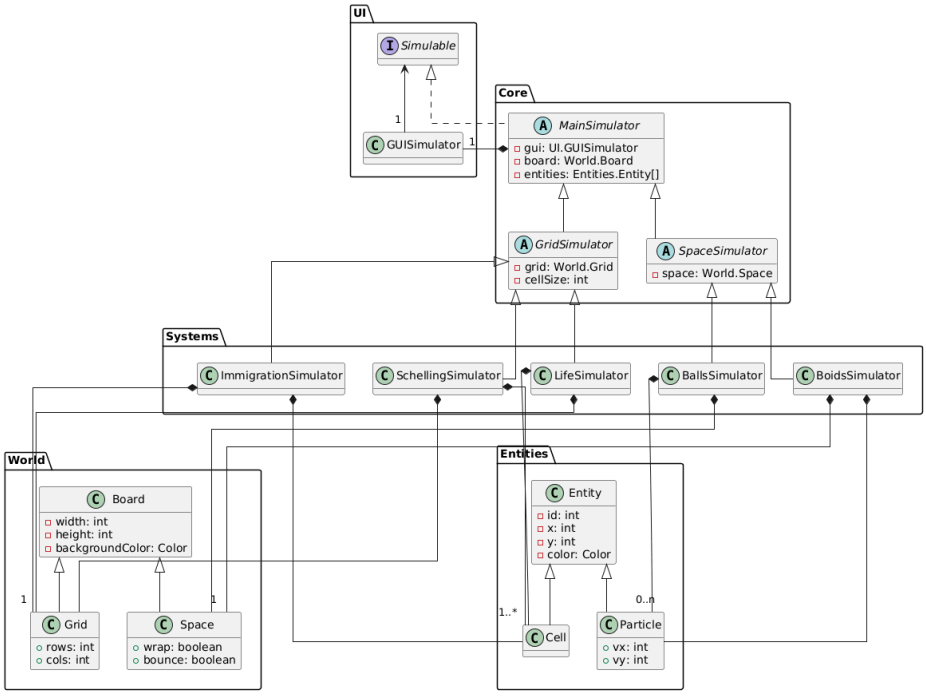


FIGURE 3 – Conception initiale des classes sans attributs. Le code et l'UML complet peuvent être consultés dans le dossier rapport/assets/.

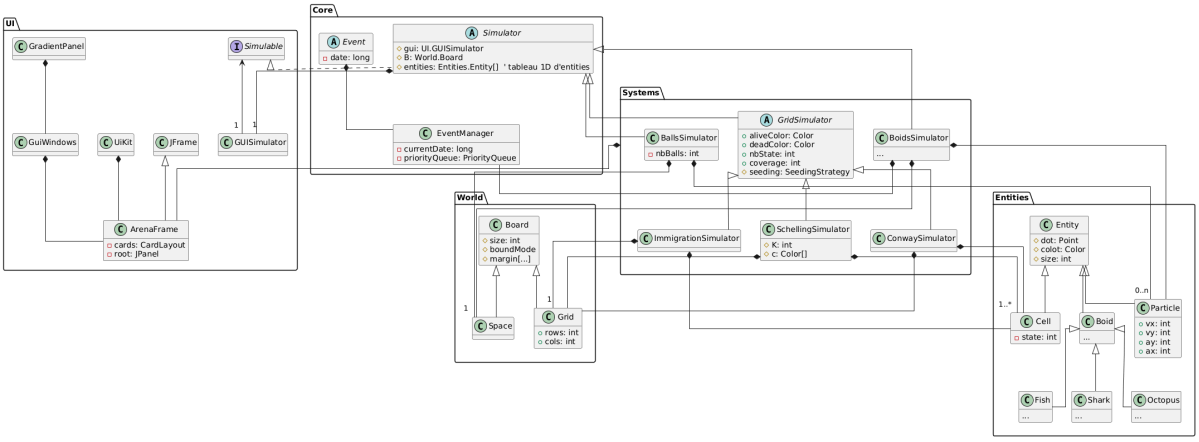


FIGURE 4 – Conception final des classes sans attributs.

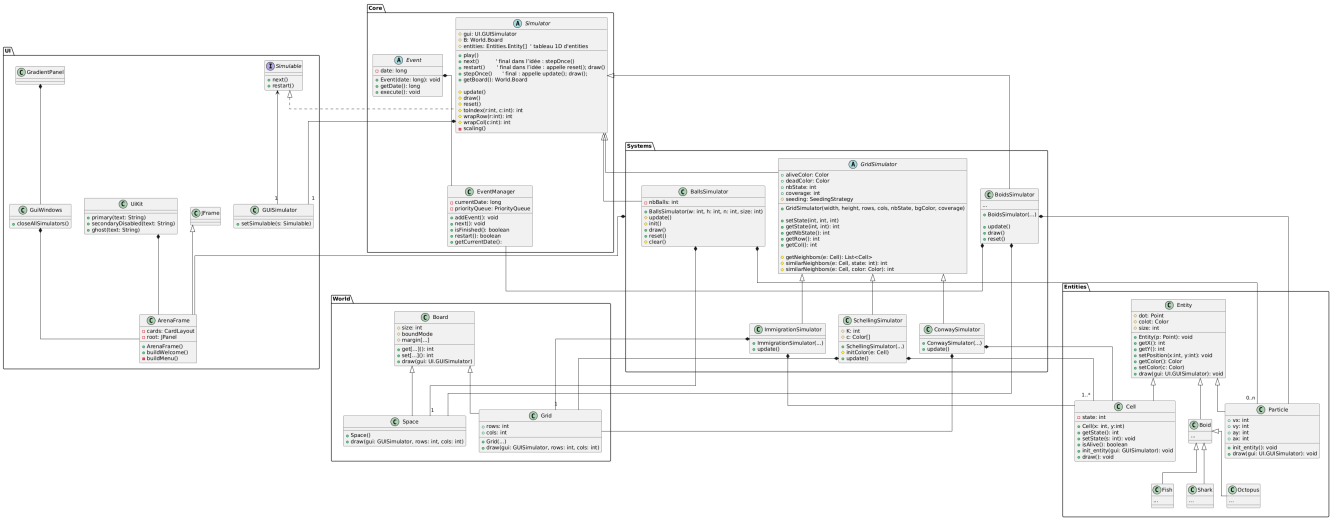


FIGURE 5 – Conception finale complète.